

CONCURRENCY AND C++

CONCURRENCY AND C++

- 1 ОСНОВНЫЕ ПОНЯТИЯ
- 2 УПРАВЛЕНИЕ ПОТОКАМИ
- 3 СОВМЕСТНОЕ ИСПОЛЬЗОВАНИЕ ДАННЫХ

ОСНОВНЫЕ ПОНЯТИЯ

1

ОСНОВНЫЕ ПОНЯТИЯ

2

CONCURRENCY – КОНКУРЕНТНОСТЬ

3

CONCURRENCY И ПАРАЛЛЕЛИЗМ

4

ИСТОРИЯ МНОГОПОТОЧНОСТИ В C++

ОСНОВНЫЕ ПОНЯТИЯ

Concurrency — параллелизм / конкурентность.

Конкурентность — это одновременное выполнение двух или более отдельных задач.

Многозадачность — это способность ОС запускать одновременно несколько приложений с помощью переключения задач.

Параллелизм — это наличие более одного вычислительного устройства (процессора или ядра), которые будут (действительно) одновременно выполнять несколько задач.

Многопоточность — это механизм ОС, когда в рамках одного процесса можно создать несколько потоков, каждый из которых будет выполнять свою задачу.

Асинхронность — это выполнение задачи таким образом, что основной поток не блокируется.

ОСНОВНЫЕ ПОНЯТИЯ

Процесс — это экземпляр выполняющейся программы, который имеет свое собственное виртуальное адресное пространство и ресурсы, такие как файловые дескрипторы и переменные окружения. Каждый процесс имеет как минимум один поток выполнения, который выполняет инструкции программы.

Поток — это отдельная единица выполнения в рамках процесса. Каждый поток имеет свой собственный стек и регистры процессора, но разделяет общее адресное пространство и ресурсы с другими потоками в рамках одного процесса.

Concurrency — параллелизм / конкурентность.

Часто путают «параллелизм» и «конкурентность». Оба термина означают одновременное выполнение, но первый на физическом уровне (параллельное исполнение нескольких процессов за счёт использования соответствующей аппаратной поддержки: процессоров, ядер), а второй — на логическом (одновременные вычисления, обеспечиваемые средствами конкретного языка и среды выполнения).

В конкурентности под одновременностью вычислений понимается не столько выполнение сразу нескольких вычислений в одно и то же физическое время (что относится к параллелизму), сколько выполнение разных взаимосвязанных вычислений за один и тот же период времени в рамках одного приложения с использованием программных средств синхронизации (в параллельных вычислениях синхронизация не нужна, в силу их логической и аппаратной независимости).

Конкурентность на уровне машины реализуется:

1. За счёт переключения задач (многозадачность).

На компьютере с одним процессором и одним ядром переключение задач ОС происходит так быстро, что создаёт иллюзию одновременного исполнения приложений для пользователя.

2. За счёт (действительно) параллельной работы многопроцессорного или многоядерного компьютера (аппаратная конкурентность).

Выполнение нескольких задач в параллельном режиме многопроцессорной системы или многоядерного процессора.

Конкурентность на уровне приложения реализуется:

1. Распределённые приложения.

Приложение делится на несколько процессов с одним потоком. Общение происходит по каналам межпроцессного обмена данными (сокеты, файлы и др.). Недостатком является то, что такую связь сложно организовать и/или она медленно работает. А преимуществом — то, что процессы можно запускать на разных компьютерах, входящих в одну сеть.

2. Многопоточные приложения.

В приложении создаётся несколько потоков. Потоки могут выполнять каждый свою последовательность инструкций. Они имеют общее адресное пространство, что даёт им лёгкий доступ к основной части данных, сокращает издержки работы по сравнению с разными процессами и снижает нагрузку на ОС. Недостатком — то, что защита общих данных от неконсистентности (согласованность и целостность) ложится на программиста.

3. Смешанный вариант.

CONCURRENCY И ПАРАЛЛЕЛИЗМ

Для многопоточного кода понятия «конкурентность» и «параллелизм» пересекаются. Оба термина относятся к одновременному выполнению нескольких задач. Но о **параллелизме** говорят, когда главной заботой становится использование имеющегося оборудования для повышения производительности массовой обработки данных. А у **конкурентности** основная задача – разделение задач, требующих одновременного решения, или оперативность реагирования.

В многопоточном приложении количество потоков может быть меньше или равно количеству ядер процессора. Тогда говорят, что количество логических потоков совпадает с количеством аппаратных потоков и независимые задачи приложения выполняются действительно одновременно. Если количество потоков больше количества ядер, то оставшая часть потоков будет распределена по ядрам ОС-ой. И потоки в рамках конкретного ядра будут выполняться «одновременно» путём переключения.

Параллелизм — это про использование имеющегося оборудования для повышения производительности массовой обработки данных.

Конкурентность — это про разделение задач, требующих одновременного решения.

>>>

Конкурентность **используется** для разделения задач в приложении и повышения производительности.

Конкурентность **не нужна**, если она приводит к отрицательному балансу выгод и затрат.

Сколько потоков может быть создано для 32-разрядных процессоров с плоской архитектурой?

Ответ: 4096

Каждый поток имеет свой стек размером в 1 Мбайт. У 32-разрядного процессора с плоской архитектурой¹ адресное пространство ограничено 4 Гбайтами. Соответственно, 4096 потоков займут всю память, не оставляя места для кода приложений и данных кучи.

В 64-разрядной системе ограничения по памяти не так существенны, но нельзя забывать и про другие накладные расходы работы ОС для потоков.

1. Плоская архитектура (Flat) – это модель памяти от нуля и до макс. с аналогичными по размеру регистрами ЦП, что позволяет адресовать всю память одним регистром. А вирт. память - это технология (алгоритм) распределения физ.памяти ОЗУ, путём замещения одних физ.страниц, другими.



Низкие издержки на запуск потоков внутри процесса и обмена данными между ними стали причиной популярности создания многопоточных приложений.

ИСТОРИЯ МНОГОПОТОЧНОСТИ В C++

В стандарте C++ 1998 г. (C99) поддержки многопоточности нет. Также не определена и формальная модель памяти, ориентированная на многопоточность. Поэтому поставщики компиляторов добавляли свои расширения языка для многопоточности. Было создано два стандарта потоков:

1. POSIX (IEEE — Institute of Electrical and Electronics Engineers)
2. MFC (Microsoft Foundation Classes)

C++11 стандарт описал:

1. Модель памяти, ориентированную на многопоточность.
2. Классы для управления потоками
(`#include <thread>`).
3. Защиту совместно используемых данных
(`#include <mutex>`).
4. Синхронизацию операций между потоками
(`#include <conditional_variable>`).
5. Низкоуровневые атомарные операции
(`#include <atomic>`).

Всё это основано на разработках многопоточности из собрания библиотеки Boost.

C++14 добавил новый тип мьютекса для защиты совместно используемых данных (`shared_mutex`).

C++17 добавил полный набор параллельных алгоритмов и новый тип `shared_mutex`.

УПРАВЛЕНИЕ ПОТОКАМИ

1

ЗАПУСК ПОТОКА

2

ЗАВЕРШЕНИЕ ПОТОКА

3

ПЕРЕДАЧА ПАРАМЕТРОВ ПОТОКУ

4

ПЕРЕДАЧА ПРАВА ВЛАДЕНИЯ ПОТОКОМ

5

STD::THREAD

6

ЛЯМБДА

ЗАПУСК ПОТОКА

Поток запускается при создании экземпляра класса `std::thread`. Объекту `std::thread` надо передать в качестве параметров функцию, функтор² или лямбда функцию, которые описывают то, что должен выполнять поток.

```
1 void print_hello();
2
3 int main() {
4     std::thread th(print_hello);
5     th.join();
6 }
7
8 void print_hello() {
9     std::cout << "Привет!";
10 }
11
```

Функция

```
1 class HelloPrinter {
2     public:
3         void operator() () const {
4             std::cout << "Привет!";
5         }
6 };
7
8 int main() {
9     HelloPrinter hp;
10    std::thread th(hp);
11    th.join();
12 }
13
```

Функтор

```
1 int main() {
2     std::thread th([]() {
3         std::cout << "Привет!";
4     });
5     th.join();
6 }
7
```

Лямбда

2. Функтор – это объект, который может вызываться как функция. Класс такого объекта должен определять функцию оператора `()`.

ЗАВЕРШЕНИЕ ПОТОКА

У каждой программы есть как минимум один поток, запускаемый средой выполнения. Он называется главным и выполняет функцию `main()`. Затем программа может запустить дополнительные потоки. При выходе из `main()` главный поток завершается. После запуска каждого дополнительного потока надо принять решение: присоединить его к главному, чтобы тот ждал его завершения и уже потом завершал `main()`, или отсоединить от главного, чтобы тот работал в фоновом режиме и его завершение легло бы на ОС. Если такого решения не принять, то деструктор `std::thread` вызовет метод `std::terminate()`.

Есть 2 способа завершения потока:

1. Метод **`join()`** присоединяет поток к текущему.
2. Метод **`detach()`** отсоединяет поток и запускает его в фоновом режиме. Здесь важно помнить, чтобы такой поток не зависел от данных вызывающего потока. Т.к. при завершении вызывающего потока они станут недействительными.

При использовании функции или функтора (а не лямбды) в качестве выполняющего кода потока может понадобиться передать параметры из вызывающего потока. Для этого будет использован конструктор `std::thread` следующего вида:

```
template<class F, class... Args>  
explicit thread(F&& f, Args&&... args);
```

Первым аргументом идёт имя функции или переменной функтора, а дальше идут аргументы, необходимые выполняемому коду. И передаются они как rvalue значения, как будто они временные. Так будет сделано даже, если ожидаемый аргумент у функции — это ссылка. Ещё одним моментом является то, что в случае неявного преобразования параметров, это преобразование будет сделано только внутри функции. Т.е. после того, как данные будут переданы в объект потока (а не перед).

Какие есть проблемы в следующем примере?

```
1 void func(int i, const std::string& str);  
2  
3 int main() {  
4     char buffer[1024] = "Привет";  
5     std::thread th(func, 1, buffer);  
6     th.detach();  
7 }  
8
```

Строка 5. `std::thread th(func, 1, buffer);` — есть вероятность, что выход из `main()` произойдёт раньше, чем `buffer` будет в новом потоке преобразован в `std::string`, что вызовет UB.

Правильно: `std::thread th(func, 1, std::string(buffer));` — использование `std::string` предотвращает обращение к недействительному указателю.

Какие есть проблемы в следующем примере?

```
1 void func(int i, std::string& str);
2
3 int main() {
4     std::string str("Привет");
5     std::thread th(func, 1, str);
6     th.join();
7 }
8
```

В отличие от предыдущего примера здесь вторым аргументом функции является ссылка на не изменяемые данные, т.е. обычная ссылка. Используемый конструктор `std::thread` не рассматривает типы аргументов, а просто копирует значения. И т.к. дальше эти значения будут передаваться как `rvalue`, а `rvalue` нельзя преобразовать к обычной ссылке, то будет ошибка компиляции. При передаче обычных ссылок их надо оборачивать в `std::ref`:

```
std::thread th(func, 1, std::ref(str));
```

3. `std::unique_ptr` — это умный указатель, обеспечивающий автоматическое управления памятью для динамически выделяемых объектов. На объект может указывать только один экземпляр `std::unique_ptr`. И когда этот экземпляр уничтожается, объект, на который он указывал, удаляется.

Какие есть проблемы в следующем примере?

```
1 void func(std::unique_ptr<MyObject>);
2
3 int main() {
4     std::unique_ptr<MyObject> obj(new MyObject());
5     std::thread th(func, obj);
6     th.join();
7 }
8
```

Перемещающий конструктор и перемещающий оператор присваивания передают права владения объектом между `std::unique_ptr`³. А исходный объект остаётся с нулевым указателем. Такое перемещение позволяет принимать объекты данного типа в качестве параметров функции или возвращать их из функции. Если исходный объект временный, то перемещение выполняется автоматически. Но если им является именованное значение, передача должна быть запрошена напрямую путём вызова `std::move`:

```
std::thread th(func, std::move(obj));
```

Какие есть проблемы в следующем примере?

```
1 void func1();
2 void func2();
3
4 int main() {
5     std::thread th1(func1);
6     std::thread th2 = std::move(th1);
7     th1 = std::thread(func2);
8     std::thread th3;
9     th3 = std::move(th2);
10    th1 = std::move(th3);
11 }
12
```

Внимательно смотрите конструкторы классов:

объект потока не может копироваться между экземплярами `std::thread`, но владение им может перемещаться.

Строка 6. Явное перемещение права собственности. У `th1` больше нет связанного потока.

Строка 7. Запускается новый поток, связанный с временным объектом. Поэтому `std::move` не требуется.

Строка 8. Создан объект `th3`. Но он не связан ни с каким потоком.

Строка 9. Право собственности на самый первый созданный поток передаётся `th3`. `th2` больше не связан с потоком.

Строка 10. Вызывается `std::terminate()`

В последней строке программы происходит ошибка.

Присвоить объекту права на новый поток можно только, если он не связан на данный момент ни с каким потоком. Чтобы корректно выполнилось `th1 = std::move(th3);` надо было перед этим дождаться завершения выполнения потока (`th1.join();`) или отсоединить его (`th1.detach();`)

Пример создания класса, гарантирующего потокам завершение до выхода из области видимости (чтобы не было вызова `std::terminate`, когда забыли сделать `join()` или `detach()`). Применён явный вызов `std::move` для передачи прав владения.

```
1 void func();
2
3 class ScopedThread {
4     std::thread th;
5     public:
6     explicit ScopedThread(std::thread th_) : th(std::move(th_)) {
7         if (!th.joinable()) {
8             throw std::logic_error("Поток отсутствует!");
9         }
10    }
11    ~ScopedThread() { th.join(); }
12    ScopedThread(const ScopedThread&) = delete;
13    ScopedThread& operator = (const ScopedThread&) = delete;
14 };
15
16 int main() {
17     ScopedThread scoped_thread{std::thread(func)};
18 }
19
```

STD::THREAD

Class `std::thread`

```
namespace std {
    class thread
    {
    public:
        // class thread::id
        class id;
        using native_handle_type = /* implementation-defined */;

        // construct/copy/destroy
        thread() noexcept;
        template<class F, class... Args>
        explicit thread(F&& f, Args&&... args);
        ~thread();
        thread(const thread&) = delete;
        thread(thread&&) noexcept;
        thread& operator=(const thread&) = delete;
        thread& operator=(thread&&) noexcept;

        // members
        void swap(thread&) noexcept;
        bool joinable() const noexcept;
        void join();
        void detach();
        id get_id() const noexcept;
        native_handle_type native_handle();

        // static members
        static unsigned int hardware_concurrency() noexcept;
    };
}
```

`std::thread::hardware_concurrency()` возвращает количество потоков, которые на данной машине действительно могут работать одновременно. Если информация недоступна, то будет возвращён 0.

`std::this_thread::get_id()` возвращает id текущего потока.

Если функция выполнения потока маленькая и используется только в одном месте, то может быть удобно записать её в виде лямбды.

```
1 int main() {  
2     int value = 1;  
3     int number = 0;  
4     std::thread th([value, &number]() {  
5         std::cout  
6             << "Внутри потока value = "  
7             << value  
8             << " number = "  
9             << number  
10            << std::endl;  
11            number++;  
12    });  
13    th.join();  
14    std::cout  
15        << "После выполнения потока number = "  
16        << number;  
17 }  
18
```

СОВМЕСТНОЕ ИСПОЛЬЗОВАНИЕ ДАННЫХ

1 ЗАДАЧА

2 RACE CONDITION

3 `STD::MUTEX`

4 DEADLOCK

5 `STD::UNIQUE_LOCK`

6 LAZY INITIALIZATION

7 ЗАДАЧА

Условие.

Есть массив чисел. Надо посчитать количество делителей у каждого числа. Использовать многопоточность.

В общем виде задача звучит так: есть (или непрерывно генерируется) набор работ, который надо выполнить.

Решение.

В простом случае, когда набор работ уже имеется, их можно разбить на почти равное количество для каждого ядра.

В этом случае у нас нет совместно используемых потоками данных, т.к. исходный вектор делится на независимые части. (`std::cout` — это совместно используемый объект, но мы его не рассматриваем.)

```
1- int main() {
2     const int jobs_count = 16;
3     std::vector<Job> jobs(jobs_count);
4     const int thread_count = std::thread::hardware_concurrency();
5     const int segment_length = (jobs_count + thread_count - 1) / thread_count;
6     std::vector<std::thread> threads;
7-    for (int id = 0; id < thread_count; id++) {
8        const int begin = id * segment_length;
9        const int end = std::min(begin + segment_length, jobs_count);
10-    threads.emplace_back(
11-        [thread_id = id + 1, begin, end, &jobs] {
12            std::cout << "Поток " << thread_id << " начал работу" << std::endl;
13-            for (int i = begin; i < end; i++) {
14                std::cout << "Работа № " << i << " начала выполняться" << std::endl;
15                jobs[i].execute();
16                std::cout << "Работа № " << i << " закончила выполняться" << std::endl;
17            }
18            std::cout << "Поток " << thread_id << " закончил работу. " << std::endl;
19        }
20    );
21 }
22- for (std::thread& th: threads) {
23     th.join();
24 }
25 }
26
```

Race condition (состояние гонки) — это ситуация в многопоточной программе, когда поведение программы зависит от того, в каком порядке выполняются операции в разных потоках относительно друг друга.

Data race (гонка за данными) — это конкретный тип состояния гонки, возникающий из-за одновременного изменения состояния объекта.

Способы справиться с состоянием гонок:

1. Мьютексы

Заключить структуру данных в механизм защиты.

2. Программирование без блокировок

Изменить конструкцию структуры данных и её инвариантов так, чтобы модификации вносились в виде серии неделимых изменений.

Пример (data race)

```
1 int main() {
2     unsigned long long summ = 0;
3     const int thread_count = 10;
4     const int max = 1'000'000;
5
6     std::vector<std::thread> threads;
7     for (int i = 0; i < thread_count; i++) {
8         threads.emplace_back([&summ] {
9             for (int i = 0; i < max; i++) {
10                 summ++;
11             }
12         });
13     }
14
15     for (std::thread& th: threads) {
16         th.join();
17     }
18
19     std::cout
20         << "Сумма равна " << summ
21         << ", а должна быть " << max * thread_count;
22 }
```

Следующие схемы показывают разницу в последовательном доступе к совместным данным и параллельном. И объясняют неожиданный результат предыдущей программы.

Последовательный доступ

Поток 1	Поток 2		Целочисленное значение
			0
прочитать значение		←	0
увеличить значение			0
записать обратно		→	1
	прочитать значение	←	1
	увеличить значение		1
	записать обратно	→	2

Параллельный доступ

Поток 1	Поток 2		Целочисленное значение
			0
прочитать значение		←	0
	прочитать значение	←	0
увеличить значение			0
	увеличить значение		0
записать обратно		→	1
	записать обратно	→	1

Пример (data race)

Проблема

```
1 void transfer(  
2     std::size_t from,  
3     std::size_t to,  
4     int amount) {  
5  
6     if (accounts[from] < amount) {  
7         throw std::runtime_error("Недостаточно денег.");  
8     }  
9     accounts[from] -= amount;  
10    accounts[to] += amount;  
11 }  
12
```

transfer переводит деньги с одного счёта на другой. И, т.о., вызывается из нескольких потоков одновременно. Каждый элемент массива accounts является совместно используемым. Если одновременно происходят переводы, связанные с одним счётом: transfer(1, 2, 100) и transfer(2, 3, 200), — то это состояние гонки и результат неопределён (UB).

>>> Нужна защита данных

Мьютекс (взаимное исключение) — это примитив синхронизации, который используется для обеспечения взаимного исключения при доступе к общим данным. Он позволяет только одному потоку за раз входить в критическую секцию кода, где происходит доступ к общим данным. Другие потоки, которые пытаются войти в критическую секцию, будут заблокированы до тех пор, пока первый поток не выйдет из нее и не освободит мьютекс.

Решение № 1

```
1 std::mutex mt;  
2 void transfer(  
3     std::size_t from,  
4     std::size_t to,  
5     int amount) {  
6  
7     mt.lock();  
8     if (accounts[from] < amount) {  
9         throw std::runtime_error("Недостаточно денег.");  
10    }  
11    accounts[from] -= amount;  
12    accounts[to] += amount;  
13    mt.unlock();  
14 }  
15
```

Решение № 1 имеет один большой недостаток: оно блокирует работу для всех случаев перевода денег.

Одновременным вызовам `transfer(1, 2, 100)` и `transfer(3, 4, 200)` не нужна блокировка, т.к. у них происходит работа с разными счетами.

>>> Надо ставить блокировку на данные, а не блок кода

Решение № 2 вводит мьютекс для каждого счёта. И, т.о., позволяет ставить блокировку именно на работу с конкретным счётом.

Для блокировки используется RAII обёртка мьютекса `std::lock_guard`, которая в деструкторе вызывает `unlock()`.

В C++17 создана улучшенная версия «охранника» в виде `std::scoped_lock`.

Решение № 2

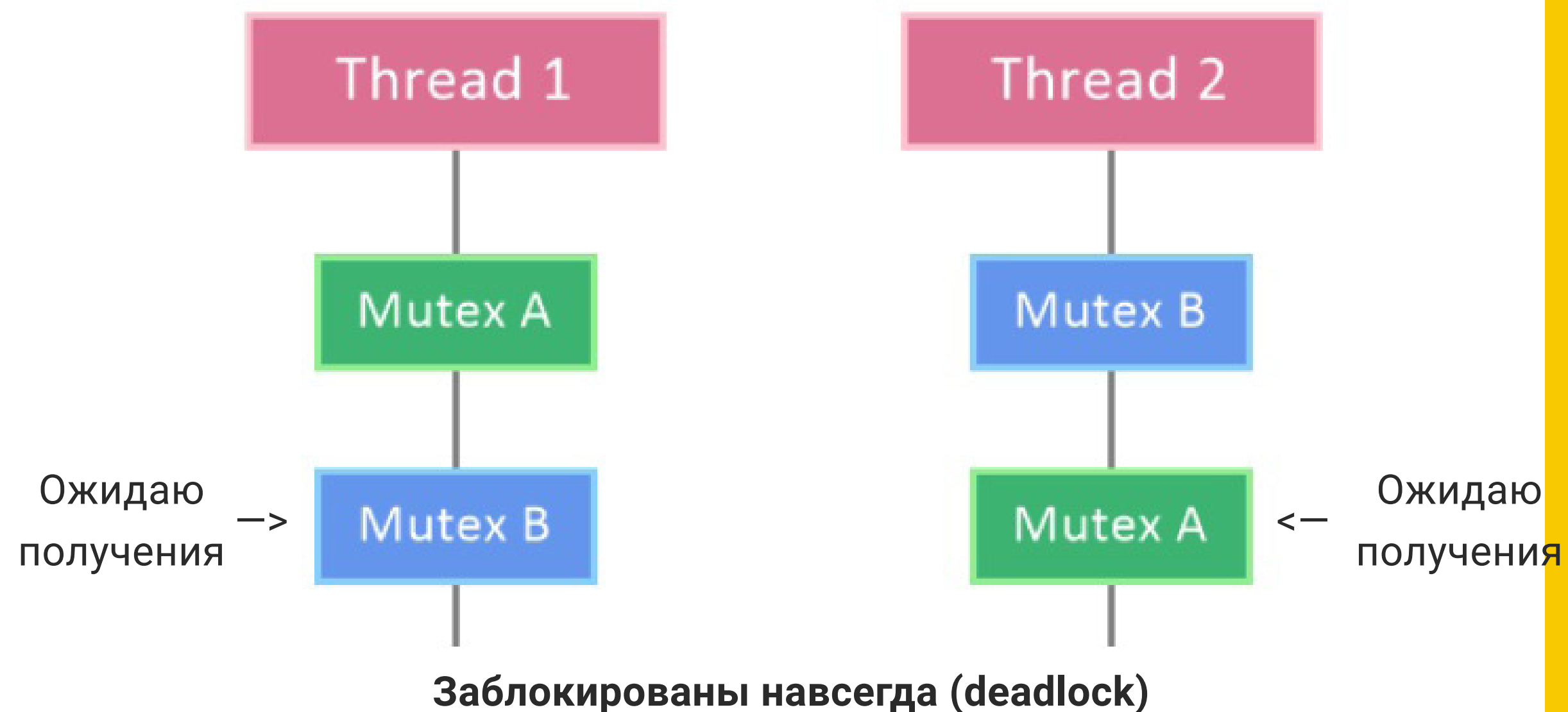
```
1 struct account {
2     std::mutex mt;
3     int32_t balance;
4 };
5
6 void transfer(
7     std::size_t from,
8     std::size_t to,
9     int amount) {
10
11     std::lock_guard<std::mutex> lg_from(accounts[from].mt);
12     std::lock_guard<std::mutex> lg_to(accounts[to].mt);
13     if (accounts[from].balance < amount) {
14         throw std::runtime_error("Недостаточно денег.");
15     }
16     accounts[from].balance -= amount;
17     accounts[to].balance += amount;
18 }
19
```

Но и у решения № 2 тоже есть серьёзные проблемы.

Рассмотрите следующий пример перевода денег:

transfer(1, 2, 100) и transfer(2, 1, 200). Что будет? >>> **Deadlock**

Deadlock (взаимная блокировка) — это ситуация в многопоточной программе, когда два или более потоков заблокированы навсегда, ожидая друг друга. Это происходит, когда несколько потоков конкурируют за доступ к общим ресурсам и блокируют друг друга, пытаясь получить их.



Решение № 3

```
1 void transfer(  
2     std::size_t from,  
3     std::size_t to,  
4     int32_t amount) {  
5  
6     if (from == to) {  
7         return;  
8     }  
9     std::lock(accounts[from].mt, accounts[to].mt);  
10    std::lock_guard<std::mutex>  
11        lg_from(accounts[from].mt, std::adopt_lock),  
12        lg_to(accounts[to].mt, std::adopt_lock);  
13    // или  
14    // std::scoped_lock guard(accounts[from].mt, accounts[to].mt);  
15    // ...  
16 }  
17
```

std::lock умеет блокировать несколько мьютексов одновременно. Также можно использовать и std::scoped_lock.

Надо помнить, что такой механизм помогает избегать deadlock только когда блокировки на объекты захватываются в одном месте. Но если это делается разобщённо, то это уже задача правильной архитектуры.

DEADLOCK

Рекомендации по обходу deadlock

1. Избегайте вложенных блокировок
2. При удержании блокировки избегайте кода, предоставленного пользователем
3. Устанавливайте блокировки в фиксированном порядке
4. Используйте иерархию блокировок

Помните: deadlock может произойти не только при установлении блокировок, но и в любой конструкции синхронизации.

`std::unique_lock` позволяет гибко владеть мьютексом, что делает его немного тяжелее и медленнее, чем `std::lock_guard`.

Его используют, когда нужна отложенная блокировка, снятие блокировки до выхода из области видимости или передача владения блокировки из одной области видимости в другую. Передача владения блокировки может передаваться путём перемещения.

`std::unique_lock` — это пример перемещаемого, но не копируемого типа.

Справа представлено ещё одно решение функции `transfer`, но уже с использованием `std::unique_lock`.

Решение № 4

```
1 void transfer(  
2     std::size_t from,  
3     std::size_t to,  
4     int32_t amount) {  
5  
6     if (from == to) {  
7         return;  
8     }  
9     std::unique_lock<std::mutex>  
10         lg_from(accounts[from].mt, std::defer_lock),  
11         lg_to(accounts[to].mt, std::defer_lock);  
12     std::lock(lg_from, lg_to);  
13     // ...  
14 }  
15
```

LAZY INITIALIZATION

Lazy initialization (ленивая инициализация) — это инициализация объекта в момент первого обращения к нему.

Есть ресурсы, инициализация которых очень затратна. И поэтому, чтобы не загружать начальную инициализацию «суперобъекта», инициализация некоторых «подобъектов» производится по требованию.

В многопоточном коде инициализация требует защиты.

Пример ленивой инициализации

```
1 - class MyResource {
2     public:
3         void process() {}
4 };
5
6 std::shared_ptr<MyResource> resource_ptr;
7
8 void process_resource() {
9     if (!resource_ptr) {
10         resource_ptr.reset(new MyResource);
11     }
12     resource_ptr->process();
13 }
14
15 int main() {
16     process_resource();
17 }
18
```

LAZY INITIALIZATION

Обычная блокировка кода

```
1  std::mutex resource_mt;
2
3  void process_resource() {
4      std::unique_lock<std::mutex> lk(resource_mt);
5      if(!resource_ptr) {
6          resource_ptr.reset(new MyResource);
7      }
8      lk.unlock();
9      resource_ptr->process();
10 }
11
```

Если защитить инициализацию просто с помощью мьютекса, то это приведёт к ненужным блокировкам (см. пример функции transfer). Для решения этой проблемы своё время был придуман паттерн «двойной проверки». Но и он имеет изъяны и в дальнейшем получил название антипаттерна.

Антипаттерн «Двойная проверка»

```
1  std::mutex resource_mt;
2
3  void process_resource() {
4      if (!resource_ptr) {
5          std::lock_guard<std::mutex> guard(resource_mt);
6          if(!resource_ptr) {
7              resource_ptr.reset(new MyResource);
8          }
9      }
10     resource_ptr->process();
11 }
12
```

После получения блокировки указатель проверяется еще раз на тот случай, если между первой проверкой и получением блокировки данным потоком инициализация была выполнена каким-нибудь другим потоком.

Проблемой здесь является то, что проверка указателя на не nullptr — это проверка на выделенность памяти, а не на инициализацию данных. Выделение памяти и инициализация данных по этой памяти — это две разные операции.

LAZY INITIALIZATION

C++11 предоставляет `std::once_flag` и `std::call_once`.

Вместо блокировки мьютекса и явной проверки указателя каждый поток может безопасно воспользоваться функцией `std::call_once`, зная, что к моменту возвращения управления из этой функции указатель будет инициализирован каким-либо потоком. Необходимые для этого данные синхронизации хранятся в экземпляре `std::once_flag`, и каждый экземпляр `std::once_flag` соответствует другой инициализации. Задействование функции `std::call_once` обычно связано с меньшими издержками по сравнению с явным использованием мьютекса.

```
1  std::once_flag resource_flag;
2
3  void init_resource() {
4      resource_ptr.reset(new MyResource);
5  }
6
7  void process_resource() {
8      std::call_once(resource_flag, init_resource);
9      resource_ptr->process();
10 }
11
```

Статические переменные и многопоточность в C++.

В C++11 инициализация `static` была решена: она выполняется только в одном потоке, и никакие другие потоки не будут продолжать выполнение до тех пор, пока эта инициализация не будет завершена. Когда нужна только одна глобальная переменная, этим свойством можно воспользоваться в качестве альтернативы `std::call_once`.

Решение с последовательным выбором работ.

Вернёмся к задаче из пункта 1 и предположим, что время выполнения работ друг от друга сильно отличается. Тогда, чтобы оптимальным образом нагрузить потоки (ядра), надо выдавать работы потокам последовательно: одну за одной по мере выполнения.

Для того чтобы поток мог выбрать следующую работу, ему надо знать сколько работ уже было выполнено и выполняется на данный момент. И здесь у нас появляется совместно используемая переменная `processed`, которая будет указывать на номер следующей работы.

Чтобы в 21 строке `current = processed++`; не было data race, надо этот блок кода заключить в блокировку.

Сроку `jobs[current].execute()`; блокировать не надо, т.к. по структуре программы обращение к конкретной работе может осуществлять только один поток.

```
1 class Job {
2     public:
3         void execute() {};
4 };
5
6 int main() {
7     const int job_count = 16;
8     std::vector<Job> jobs(job_count);
9     const int thread_count = std::thread::hardware_concurrency();
10
11     std::mutex mt;
12     std::vector<std::thread> threads;
13     int processed = 0;
14
15     for (int i = 0; i < thread_count; i++) {
16         threads.emplace_back([&jobs, &mt, &processed] {
17             while (true) {
18                 int current;
19                 {
20                     std::lock_guard<std::mutex> lk(mt);
21                     current = processed++;
22                 }
23                 if (current >= jobs.size()) {
24                     break;
25                 }
26                 jobs[current].execute();
27             }
28         });
29     }
30
31     for (std::thread& th: threads) {
32         th.join();
33     }
34 }
35
```